BYOP – Bring Your Own Project

Project Report

Chess AI Engine with Minimax Search and Alpha-Beta Pruning

Name:Yash Sharma

Registration Number: 25BAI10305

Course: Fundamentals of AI and ML

1. Introduction and Problem Statement

Chess has long been recognized as one of the oldest and most influential strategy games, and since the late 1950s it has been a prominent problem within artificial intelligence. In order to create a program capable of playing chess, the system must be able to anticipate future moves through evaluation of potential actions, evaluation of good and bad positions, as well as making intelligent decisions even when the potential outcome of an action is uncertain. All of these represent classic AI problem-solving challenges.

The main question I wanted to address in my project was; can I create a fully functioning Chess AI from scratch using only the concepts learned in this course, without the use of pre-constructed AI libraries? The answer to this question is yes. I created a Chess engine in Python that plays a complete and legal game of Chess against human players only through console interaction using both a Minimax algorithm and Alpha-Beta pruning mechanism to decide the next best move in accordance to legal game Play. The Chess board used for my engine has row and column labels that make it easy to identify pieces on the board, and a player is only needed to enter their move using standard Chess notation via keyboard input to play.

The difficulty in solving this problem is due to the fact that chess engines do not solely deal with playing chess; they rely on similar techniques (tree searching, heuristic scoring and pruning) used by robotics, medical decision-making, logistics and financial planning. Therefore, by doing this project I will have applied what I learned in class to create a real, interactive, testable product from the start.

2. Why This Problem Matters

Chess has played a huge role in creating the technology behind artificial intelligence. AI algorithms were used with chess computers and eventually helped machines to defeat the world's best players. Learning about these algorithms can give you an insight into the history of AI and help build your understanding of modern AI.

From a learning standpoint, this project is a great illustration of how the concept of adversarial search plays into creating good decisions for an AI that is under attack from another agent. The Minimax algorithm is the most widely accepted solution to the adversarial search problem. The Minimax searches all possible paths of every move and then determines which will provide the best outcome for the AI, assuming its opponent plays well.Alpha-Beta pruning is an enhancement of the Minimax algorithm that helps eliminate unnecessary portions of the search tree so that AI can execute its decision without having to evaluate the entire search tree, which significantly reduces the time required to execute a decision.

One of the best ways to see how this entire process is used in determining a chess move is to watch and see how AI arrives at a chess move in real-time while evaluating thousands of possible chess positions to select the best move for the AI. That makes the AI's reasoning visible and easy to understand, which is something a lot of AI projects do not offer.

3. Approach and Solution Design

3.1 Technology Stack

All of the code in this project has been written in Python and utilizes a single external library (python-chess) to handle all chess rules, including legal moves, when games are finished, detecting checkmate/stalemate, etc. By using an external library for all game rules, I was able to concentrate solely on writing an AI for chess and did not need to spend time coding rules of chess myself. Additionally, since the game is played in the terminal, there are no graphical components necessary, making it simple to run on any computer with a terminal available.

Project Structure

The project is organized into two separate files: engine.py (containing all AI logic - board scoring function, recursive search function, function that selects the next move) and main.py (containing the game loop, function for printing the displayed chess board, handling user input, turn management). Main.py calls engine.py to obtain the AI's move; however, main.py has no code that contains any AI logic. By separating the AI and the user interface from each other, the code is therefore much cleaner and easier to maintain. Should someone want to add a graphical interface to the game, the only file that will need to be changed is main.py; there will be no need to modify the engine.py file.

```python
def runSearch(board, depth, alpha, beta, isMaximizing):
    Click to collapse the range.
    with Alpha-Beta pruning.
    It recursively checks future moves to find the best path.
    """
    if depth == 0 or board.is_game_over():
        return getPositionRating(board)

    if isMaximizing:
        bestScore = -1000000
        for move in board.legal_moves:
            board.push(move)
            evalResult = runSearch(board, depth - 1, alpha, beta, False)
            board.pop()

            # Updating the best score and the alpha threshold
            bestScore = max(bestScore, evalResult)
            alpha = max(alpha, evalResult)

            # Pruning: stop searching this branch if it's already worse
            if beta <= alpha:
                break
        return bestScore

    else:
        bestScore = 1000000
        for move in board.legal_moves:
            board.push(move)
            evalResult = runSearch(board, depth - 1, alpha, beta, True)
            board.pop()

            # Updating the best score and the beta threshold
            bestScore = min(bestScore, evalResult)
            beta = min(beta, evalResult)

            # Pruning logic
            if beta <= alpha:
                break
        return bestScore
```

```python
def getPositionRating(board):
    """
    This function looks at the board and gives a score.
    Positive score = White is winning.
    Negative score = Black is winning.
    """
    if board.is_checkmate():
        # If it's White's turn and they are in mate, Black wins (-1M)
        if board.turn == chess.WHITE:
            return -1000000
        else:
            return 1000000

    currentScore = 0
    # Loop through each piece type and tally up the material
    for pieceType, value in pieceValues.items():
        whiteCount = len(board.pieces(pieceType, chess.WHITE))
        blackCount = len(board.pieces(pieceType, chess.BLACK))
        currentScore += (whiteCount - blackCount) * value

    return currentScore
```

Figure 1 — engine.py: complete AI logic with scoring, search, and move selection

```python
def startChessMatch():
    chessBoard = chess.Board()
    print("--- CHESS ENGINE v1.2 (Labeled Interface) ---")
    print("AI starts as White. Use UCI format (e.g., e7e5) to play.")

    while not chessBoard.is_game_over():
        # We now use our custom labeled board instead of the basic print
        printLabeledBoard(chessBoard)

        if chessBoard.turn == chess.WHITE:
            print("AI is calculating...")
            # Using 'selectMove' from our humanized engine.py
            aiMove = engine.selectMove(chessBoard, 3)
            chessBoard.push(aiMove)
            print(f"AI Move: {aiMove}")
        else:
            try:
                userMoveInput = input("Your move (Black): ").strip()
                if userMoveInput.lower() in ['exit', 'quit']:
                    break

                legalUserMove = chess.Move.from_uci(userMoveInput)
                if legalUserMove in chessBoard.legal_moves:
                    chessBoard.push(legalUserMove)
                else:
                    print("Error: Illegal move! Check the coordinates.")
            except:
                print("Error: Invalid format. Please use UCI (like 'd7d5').")

    print("\n--- Match Finished ---")
    print(f"Final Outcome: {chessBoard.result()}")
```

Figure 2 — main.py: board display, user input handling, and turn management

3.3 Board Display — printLabeledBoard()

This version's biggest change is the labeled board display. Rather than simply printing the raw board, the printLabeledBoard function in main.py produces a board that has column letters (a - h) at both the top and bottom of the board and row numbers (1 - 8) down each side of the board. This makes it much easier for a human player to read the board and type in the proper move without having to count squares manually.

```python
def printLabeledBoard(board):
    """
    This function adds rank and file labels to the board output
    so it's easier for humans to read in the terminal.
    """
    # Column headers
    print("\n     a b c d e f g h")
    print("   +-----------------+")

    # Split the board string into individual rows
    boardLines = str(board).split('\n')

    # Print each row with its rank number (8 down to 1)
    for index, line in enumerate(boardLines):
        rankNumber = 8 - index
        print(f" {rankNumber} | {line} | {rankNumber}")

    print("   +-----------------+")
    print("     a b c d e f g h\n")
```

Figure 3 — Labeled board output in the terminal (a-h columns, 1-8 rows)

3.4 Scoring the Board — getPositionRating()

The getPositionRating function determines who is ahead at any given time in the game by adding the total value of White's pieces and subtracting from that the total value of Black's pieces. The result will be positive if White is ahead, negative if Black is ahead, or 0, which would indicate the position is approximately equal.

The point values for each piece are based on standard chess theory:

Chess Piece	Point Value
Pawn	100
Knight	320
Bishop	330
Rook	500
Queen	900
King	20,000 (symbolic)

```
# My custom point system for evaluating the board
# Standard values: Pawn=100, Knight/Bishop=320, Rook=500, Queen=900
pieceValues = {
    Pawn: 100,
    Knight: 320,
    Bishop: 330,
    Rook: 500,
    Queen: 900,
    King: 20000
}
```

Figure 4 — getPositionRating(): material-based board evaluation function

The function also checks for checkmate first. If the current player is in checkmate, the function returns positive or negative one million. This makes sure the AI always treats checkmate as the most important outcome.

3.5 The Search Algorithm — runSearch()

The runSearch function is beyond a doubt the "heart" of our engine. It works straightforwardly; it is a recursive function that looks at all legal moves, performs the legal move on the board, calls itself to look at what the opponent can do to reply to that move and the process repeats until the pre-determined depth is achieved. When it reaches the lowest point of recursion, the getPositionRating function is used to determine how well the position evaluated and the process goes back up the tree keeping track of the best move throughout.

The primary mechanism for determining how to proceed is through the alternate mindsets of the AI. On the AI's (maximising player's) turn, the allowable move is whichever move returns the highest score. On the human player’s turn (the minimising player), the assumption is that the opponent will also perform the allowable move that returns the lowest score..

```python
def runSearch(board, depth, alpha, beta, isMaximizing):
    """
    The main Minimax algorithm with Alpha-Beta pruning.
    It recursively checks future moves to find the best path.
    """
    if depth == 0 or board.is_game_over():
        return getPositionRating(board)

    if isMaximizing:
        bestScore = -1000000
        for move in board.legal_moves:
            board.push(move)
            evalResult = runSearch(board, depth - 1, alpha, beta, False)
            board.pop()

            # Updating the best score and the alpha threshold
            bestScore = max(bestScore, evalResult)
            alpha = max(alpha, evalResult)

            # Pruning: stop searching this branch if it's already worse
            if beta <= alpha:
                break
        return bestScore

    else:
        bestScore = 1000000
        for move in board.legal_moves:
            board.push(move)
            evalResult = runSearch(board, depth - 1, alpha, beta, True)
            board.pop()

            # Updating the best score and the beta threshold
            bestScore = min(bestScore, evalResult)
            beta = min(beta, evalResult)

            # Pruning logic
            if beta <= alpha:
                break
        return bestScore
```

Figure 5 — runSearch(): recursive Minimax with Alpha-Beta pruning

3.6 Alpha-Beta Pruning

Due to no optimizations, the growth of the search tree is enormous; at a depth of three, and 30 possible moves per position, there are about 27,000 positions to be evaluated at that depth; by depth five, you would have over 24 million. Alpha-beta pruning solves this problem by maintaining a record of the best scores that have been discovered for both players. When the algorithm discovers a branch that couldn't have a better score than what has already been found for either player, they stop searching that branch altogether and move on.

In the code this takes place in the runSearch function where the use of alpha and beta variables comes into play. When beta is greater than or equal to alpha, the current branch is pruned by issuing a break statement. This can reduce the number of positions that have been evaluated to approximately 50%

3.7 Move Selection — selectMove()

When it's the AI's turn, main.py calls the selectMove function to find out what the best move should be. The function loops through all of the available legal moves and calls the runSearch function for each one to see what score it got back. After looping through all of the legal moves, the function picks the move with the highest returned score. The default value for the depth is 3 because this provides a good balance between thinking time and the strength of the move.

```python
def selectMove(board, currentDepth):
    """
    This is the entry point called by main.py.
    It picks the move that has the highest 'runSearch' score.
    """
    bestMoveFound = None
    highestValue = -1000000

    for move in board.legal_moves:
        board.push(move)
        # We start the recursive search one level deeper
        moveValue = runSearch(board, currentDepth - 1, -1000000, 1000000, False)
        board.pop()

        if moveValue > highestValue:
            highestValue = moveValue
            bestMoveFound = move

    return bestMoveFound
```

Figure 6 — selectMove(): entry point that picks the best move using runSearch()

4. Key Decisions Made During Development

By adding a labeled display for the board, the original version was printed without any kind of aide to use. Adding the function printLabeledBoard that labels ranks and files made the game actually able to be played and provided for a more professional look for the game.

Each function in engine.py has been renamed to provide a more descriptive name that relates directly to what that function does. The new names of the functions, such as getPositionRating, runSearch, and selectMove establish a direct relationship to the functionality of the respective function without needing to read code to understand what the function does.

I purposely left the evaluation function material-only to maintain the focus of the project on to the search algorithm itself. Adding positional scoring to the evaluation function is documented in the README as a clear next step of the project.

By setting depth = 3 as default, I have created a good stable environment for both response time and playing strength. The depth can easily be changed in main.py script for anyone wanting stronger play.

All logic used for engine (AI) functions are located within engine.py, all display/input code is located in main.py. If something goes wrong, it is very clear which source file to look at for debugging purposes.

5. Challenges Faced

The following items were difficult to achieve and required a lot of time and patience:

1. Correctly alternating between the maximization and minimization sides during the recursive calls in the minimax algorithm. I had to perform many successful tests using pre-known position to confirm that the alternation worked as intended.

2. Incorrectly returning results when using alpha-beta pruning. A small mismatch in the pruning condition may cause the algorithm to discard a branch that should have been searched. This process required extreme attention to detail in order to get the conditions updated using alpha (the highest value will map to the maximizing player) and beta (the lowest value will map to the minimizing player) exactly correct.

3. The engine takes longer to perform at greater depths than previously. Even when using alpha-beta pruning, performance will degrade significantly once you pass depth level 4 if a position is complex. For this reason, the default depth of 3 was selected because it is fast enough to give the impression of being responsive.

4. The AI will not prioritize checkmate in an early version. It would frequently select to take a piece rather than attempt to place the opposing player in checkmate. A simple adjustment of moving the checkmate detection to before the rating of the position was calculated resolved this issue.

5. Human-readable board. The default output from the python-chess library does not have any identifiers on the board, which made it almost impossible for a user to determine what square is located where. The printLabeledBoard() function resolved this problem.

6. Results and Testing

Several matches between the engine and humans were conducted in order to evaluate the engines operation. The following lists the highlights from these matches:

- When playing at a depth of 3, the AI either plays legal or sensible moves throughout the game. It captures any unprotected pieces and occasionally recognizes a basic tactical idea such as capturing with a fork.
- The AI can correctly evaluate checkmates. If the checkmate is one it can reach within its depth of move selections, then the AI will checkmate the user.
- The AI correctly handles all of the ways to draw a chess game: stalemate, insufficient material, three-time repeat, and fifty-move rule.
- The labeled board displays enable an easy following of the game with its coordinates on the outside of the board
- The input validators ensure that all input is either an invalid UCI format or an illegal chess move, and gives a logical explanation for what the invalid input is.

Overall, the AI plays at about the beginner to intermediate level; it will play a legal move and win a piece whenever possible, but it lacks the sophistication of playing positional chess, which is what can be expected given that it uses a material-only evaluation function, and will be improved upon in the future.

7. What I Learned

This project taught me things that I simply could not have learned from reading notes or doing exercises alone.

What I learned most from this project is that adversarial search is much more complicated than I thought. Before this project I understood Minimax as a concept but after working through the coding and debugging process I understand it much more thoroughly. I now can trace the recursion through my head, predict the output of the algorithm at any position I can think of, and articulate the reason or reasoning behind the algorithm's decision-making ability.

I also learned that the evaluation function is where all the intelligence actually is. The search algorithm is simply a framework to explore the search tree. The AI's intelligence and lack of intelligence is entirely a function of how well the scoring function can identify good and bad positions. If I had a better scoring function than I do now, with the same level of deepness to the search tree, my AI would be able to play much stronger chess. This is a very important lesson in designing AI, not just for chess but for designing all AIs.

Finally, building an interactive experience for a user is a much different experience than writing code to calculate a numeric value. Watching the AI make its first move, identifying a capture I did not catch, and watching it avoid a trap I set for it made this whole project feel much more real to me than any textbook problem has.

9. Conclusion

This project demonstrates that it is possible to develop an interactive chess AI from scratch based solely on the foundation of the coursework for Fundamentals of AI and ML course. The algorithm for our chess engine operates on the principles of adversarial search via the Minimax algorithm while also optimizing its performance via Alpha-Beta pruning as well as judging board positions using a score based on the material value of pieces. The end result of this project is an engine capable of playing chess and making logical moves against a human opponent while offering a rewarding experience for everyone involved.

The codebase for this project has been organized in order to promote ease-of-use by properly structuring the code using descriptive names and comments, labeling the chessboard such that all players can understand what is happening, and clearly outlining installation instructions in the README file. Each design decision was made intentionally, each challenge was solved appropriately, and the lessons learned throughout this project will be retained for many years to come.

What really resonates with me about this project is how much the abstract concepts we learned throughout the class are now tied to something tangible after having built this project: adversarial search, heuristic evaluation, pruning trees; these used to be topics on my exams but now they're actual working tools through which I successfully built this chess program. This is the greatest value a project of this magnitude can provide me with.

References

- Wikipedia – Minimax algorithm – en.wikipedia.org/wiki/Minimax
- Wikipedia – Alpha-Beta pruning – en.wikipedia.org/wiki/Alpha-beta_pruning
- Chess programming Wiki – Evaluation – www.chessprogramming.org/Evaluation
- GitHub Repository – https://github.com/Yash0311-GLP/CHESS_AI